

Administrateur

- **Gérer les utilisateurs.**
- **Gérer les rôles.**
- **Gérer classes, matières et paramètres système.**
- **Consulter les statistiques globales.scribd+1**

Enseignant

- **Créer, modifier, supprimer et publier des QCM.**
- **Ajouter les questions manuellement.**
- **Importer ou générer des questions via IA.**
- **Créer une session live.**
- **Générer PIN et QR Code.**
- **Lancer le quiz.**
- **Consulter les résultats et exporter les statistiques**

Étudiant

- **Se connecter.**
- **Rejoindre un quiz via PIN/QR.**
- **Répondre aux questions.**
- **Voir score, classement, badges et historique.**

Invité

- **Rejoindre un quiz sans compte.**
- **Choisir un pseudonyme.**
- **Répondre aux questions.**
- **Voir score et classement.**

Structure du projet

Tu peux organiser ton application ainsi :

- **Web app React : admin + teacher.**
- **Mobile app React Native : student + guest.**
- **Backend Spring Boot : logique métier.**
- **WebSocket : live quiz.**
- **PostgreSQL : données.**
- **IA service : génération automatique de QCM.**

Schéma logique simple

- **Admin / Teacher → Web React**
- **Student / Guest → Mobile React Native**
- **Toutes les requêtes → Spring Boot**
- **Sessions live → WebSocket**
- **Données → PostgreSQL**
- **PDF vers QCM → IA**

Diagrammes à produire

Je te liste les principaux diagrammes + ce qu'ils doivent contenir pour ton sujet.

1 Diagramme de cas d'utilisation (UML)

Acteurs :

- Administrateur
- Enseignant
- Étudiant
- (optionnel) Service IA de génération de quiz (acteur secondaire / système externe)

Use cases principaux par acteur :

- Administrateur
 - Gérer utilisateurs (CRUD enseignant/étudiant)
 - Gérer matières, niveaux, classes
 - Consulter statistiques globales
- Enseignant
 - S'authentifier
 - Gérer son profil
 - Créer / modifier / supprimer QCM
 - Gérer banque de questions (import Excel/Word/PDF, duplication)
 - Programmer une session (mode Live, Challenge, Examen, Entraînement)
 - Lancer un quiz en direct
 - Consulter résultats / statistiques / tableaux de bord
 - Exporter résultats (PDF/Excel/CSV)
 - Générer un QCM à partir d'un document (PDF/Word/PPT) via IA
- Étudiant
 - S'authentifier
 - Rejoindre une session via PIN ou QR Code
 - Répondre au quiz
 - Voir score, classement, historique

- Recevoir badges/récompenses

Tu peux t'inspirer des use cases classiques "online examination system" et les adapter à ton contexte (modes Live/Kahoot + IA). [youtube](#)

2.2 Diagramme de classes (UML)

Classes métier principales (côté backend) :

- Utilisateur
 - id, nom, prénom, email, motDePasseHash, rôle, dateCréation, actif
- Rôle
 - id, nom (ADMIN, ENSEIGNANT, ETUDIANT), permissions
- Enseignant (extends Utilisateur ou association 1–1)
 - spécialité, grade, etc.
- Étudiant (extends Utilisateur ou association 1–1)
 - matricule, niveau, classeRef
- Classe
 - id, nom, niveau, annéeScolaire
- Matière
 - id, nom, description, enseignantRef
- QCM
 - id, titre, description, durée, niveau, difficulté, matièreRef, mode (LIVE, CHALLENGE, EXAMEN, ENTRAINEMENT), dateCréation, propriétaire (enseignantRef), sourceGeneration (MANUEL, IA_PDF, IA_WORD, IA_PPT)
- Question
 - id, enonce, type (CHOIX_UNIQUE, CHOIX_MULTIPLE, VRAI_FAUX, IMAGE, MEDIA), temps, imageUrl, mediaUrl, explication, ordre, qcmRef
- Réponse
 - id, texte, correcte (bool), ordre, questionRef
- SessionQuiz
 - id, qcmRef, dateDebutPlanifiee, dateDebutReelle, dateFin, codePIN, qrCodeToken, mode (LIVE, CHALLENGE, EXAMEN, ENTRAINEMENT), statut (PLANIFIEE, EN_COURS, TERMINEE)
- Participation
 - id, sessionRef, etudiantRef ou pseudo (pour mode invité), scoreTotal, tempsTotal, rangFinal, dateParticipation
- RéponseEtudiant
 - id, participationRef, questionRef, choixSelectionnes, estCorrect, tempsReponse
- StatistiqueQuestion
 - questionRef, tauxReussite, nbTentatives, tempsMoyen
- Badge
 - id, nom, description, condition (score > X, nbSessions, etc.)

- BadgeEtudiant
 - id, etudiantRef, badgeRef, dateAttribution
- Notification
 - id, utilisateurRef, titre, message, type (EMAIL, IN_APP), date, lu
- HistoriqueAction
 - id, utilisateurRef, typeAction, details, date
- Export
 - id, type (PDF, EXCEL, CSV), date, enseignantRef, sessionRef
- DocumentSource (pour module IA)
 - id, type (PDF, WORD, PPT), urlStockage, status (UPLOADED, PARSED, INDEXED), langue
- QuizGenerationJob
 - id, documentSourceRef, enseignantRef, dateDemande, status (EN_ATTENTE, EN_COURS, TERMINE, ECHEC), logs

1. Acteurs dans les cas d'utilisation

Pour ton diagramme de cas d'utilisation, tu peux définir :

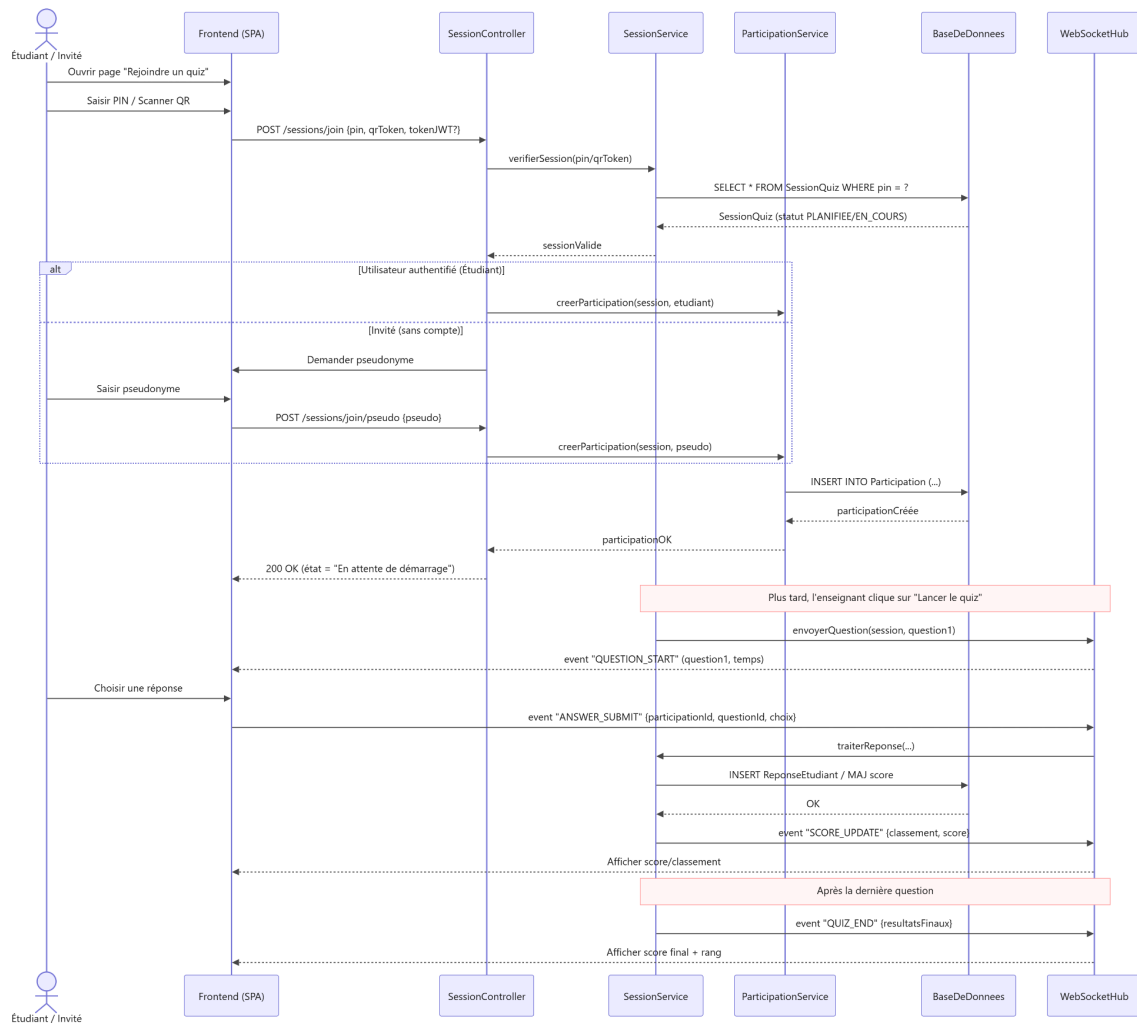
- **Administrateur**
- **Enseignant**
- **Étudiant authentifié** (qui a un compte)
- **Participant invité** (ou "Joueur invité")

Le **Participant invité** représente toute personne qui :

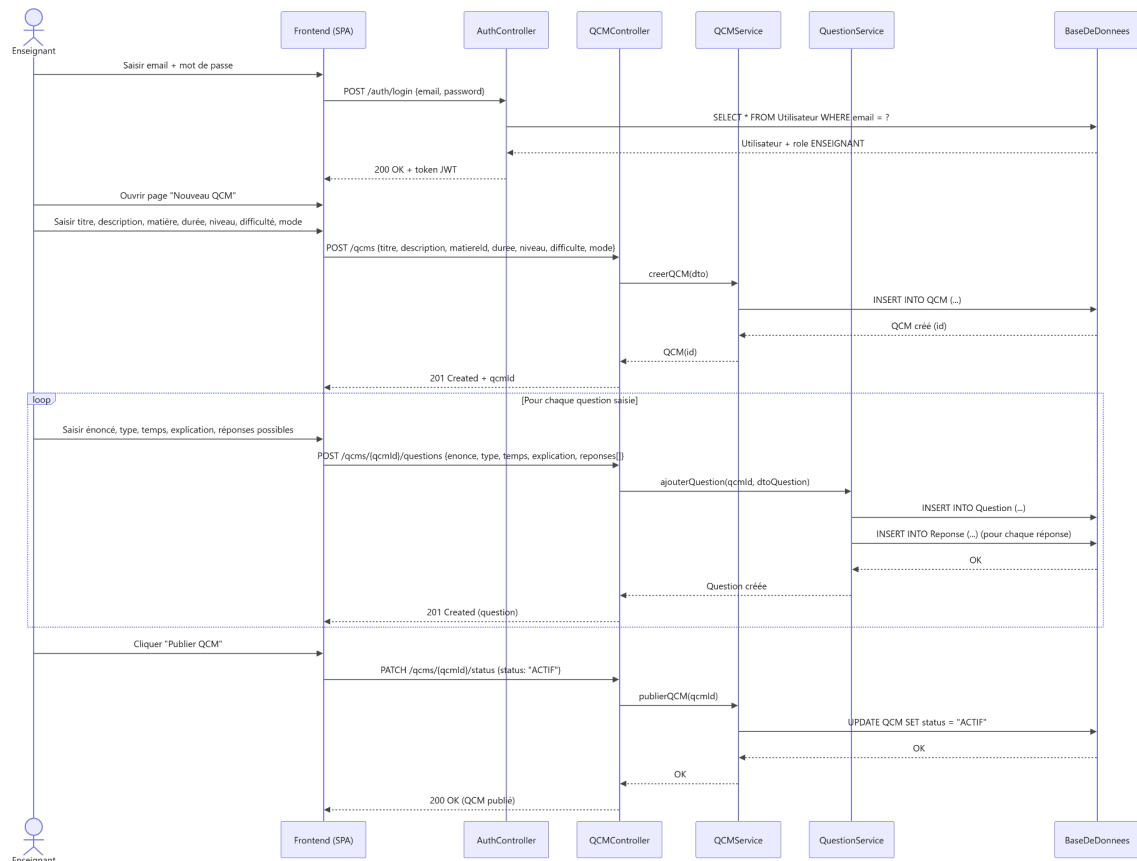
- accède à l'URL de participation,
- saisit le **code PIN** ou scanne le **QR Code**,
- choisit un **pseudonyme**,
- répond au quiz et voit son score/classement, sans être obligé de créer un compte, comme sur Kahoot où on peut rejoindre un jeu en tant qu'invité avec un simple PIN ou un QR.

3. Use case – Vue Étudiant + Invité + IA (participation, historique, auto-entraînement)

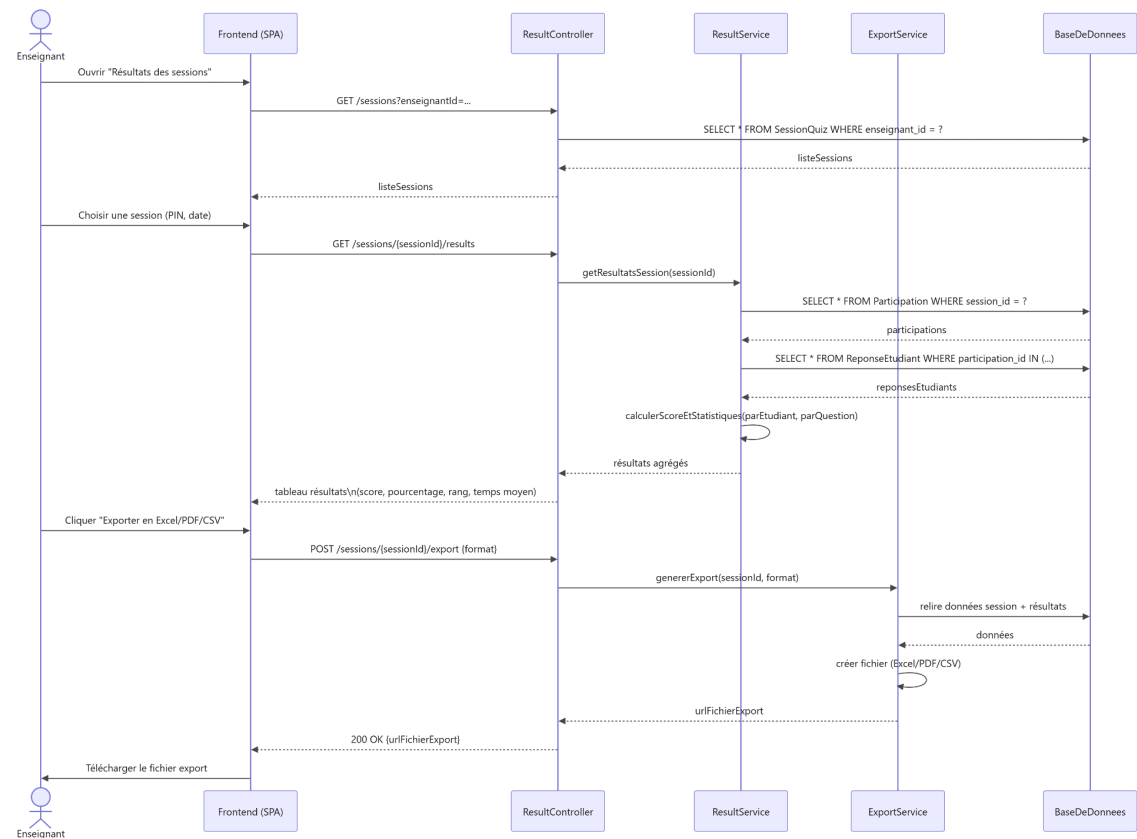
1 Diagramme de séquence – “Rejoindre un quiz via PIN/QR”



2. Séquence – Création manuelle de QCM par enseignant



3 Séquence – Consultation des résultats et export par enseignant



4 Séquence – Génération de QCM via IA à partir d'un PDF

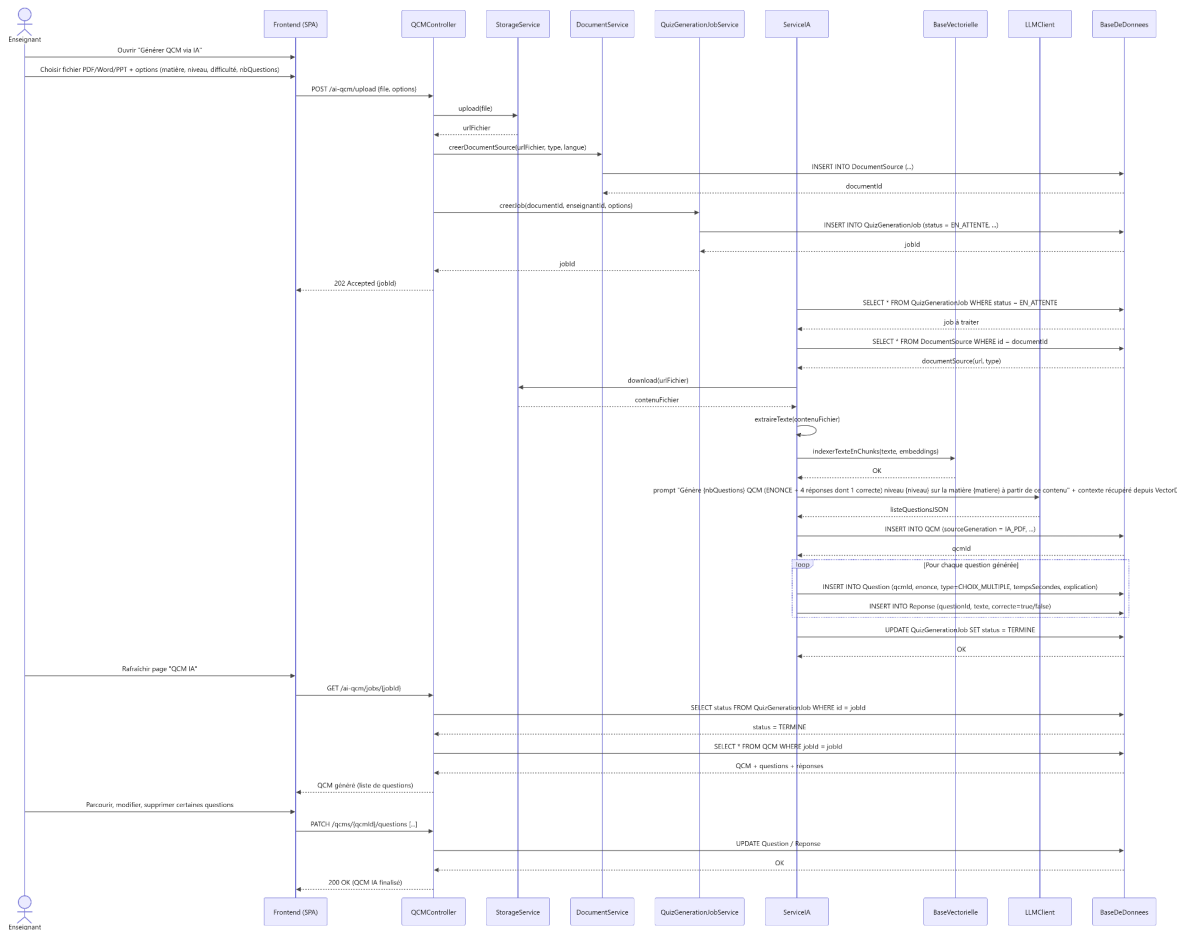
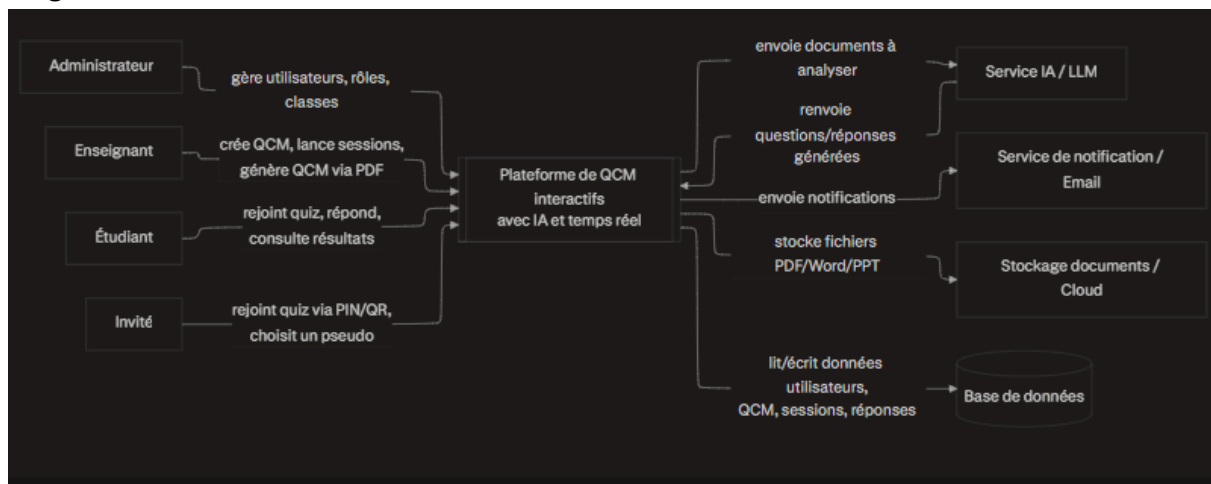


diagramme de contexte



ERD proposé

Pour ton système de QCM interactifs avec IA, les entités essentielles sont :

Utilisateur, Rôle, Matière, QCM, Question, Réponse, SessionQuiz, Participation, RéponseEtudiant, DocumentSource, QuizGenerationJob.

Ce type de structure correspond bien aux modèles ER de systèmes de quiz/examen en ligne, où l'on relie utilisateurs, quizzes, questions, résultats et réponses.

Relations principales

- Un **Utilisateur** peut avoir un ou plusieurs rôles.
- Un **Enseignant** crée plusieurs **QCM**.
- Un **QCM** contient plusieurs **Questions**.
- Une **Question** contient plusieurs **Réponses**.
- Un **QCM** peut générer plusieurs **SessionQuiz**.
- Une **SessionQuiz** contient plusieurs **Participations**.
- Une **Participation** contient plusieurs **RéponseEtudiant**.
- Un **DocumentSource** est lié à un **QuizGenerationJob** pour la génération IA.
- Un **QuizGenerationJob** peut produire un **QCM**

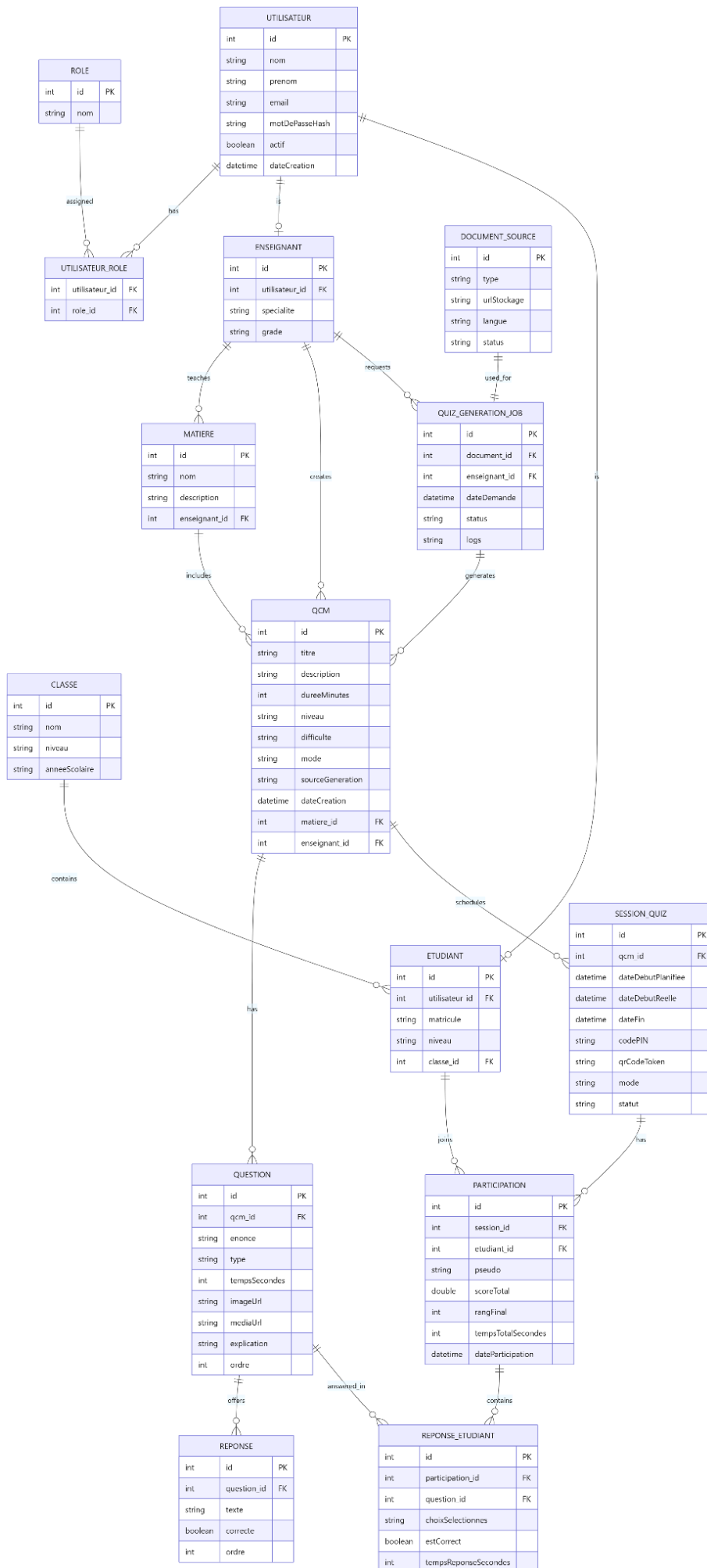
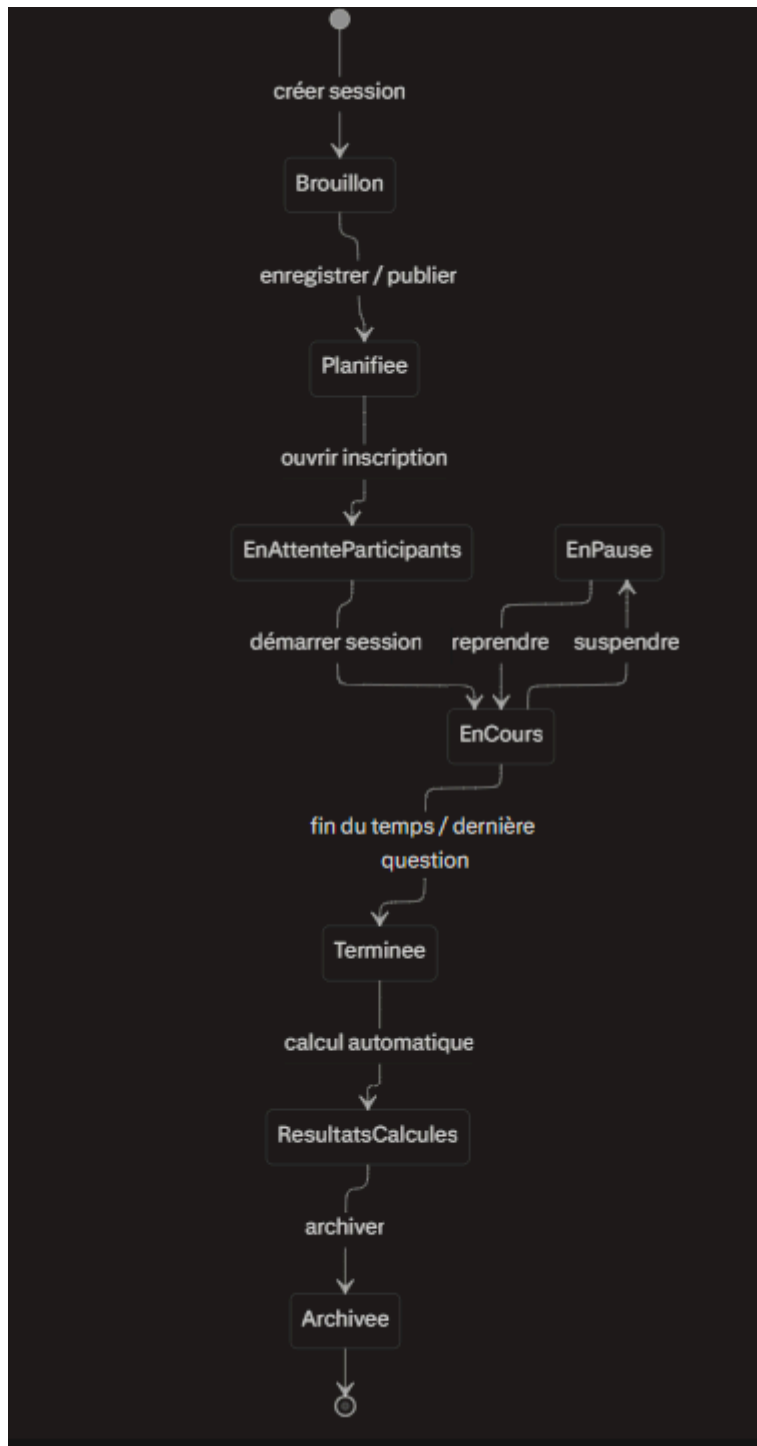


Diagramme d'activité précis — Participer à une session live



Diagramme d'état précis — SessionQuiz



Exigences fonctionnelles

1. Le système doit permettre à un administrateur de gérer les utilisateurs, les rôles, les classes et les matières.
2. Le système doit permettre à l'enseignant de créer, modifier, supprimer et publier des QCM.
3. Le système doit permettre à l'enseignant d'ajouter des questions manuellement ou d'importer une banque de questions.
4. Le système doit permettre la génération automatique d'un QCM à partir d'un document PDF, Word ou PPT via le module IA.[scribd](#)
5. Le système doit permettre de créer une session de quiz avec un code PIN et un QR Code.
6. Le système doit permettre à un étudiant ou à un invité de rejoindre une session de quiz via PIN ou QR Code.
7. Le système doit permettre à un invité de participer en choisissant un pseudonyme.[scribd](#)
8. Le système doit permettre aux participants de répondre aux questions en temps réel.[scribd](#)
9. Le système doit corriger automatiquement les réponses et calculer les scores.[scribd](#)
10. Le système doit afficher le classement, le score final et les résultats détaillés.[scribd](#)
11. Le système doit permettre à l'enseignant de consulter les statistiques des sessions et des étudiants.
12. Le système doit permettre l'export des résultats au format PDF, Excel ou CSV.[scribd](#)
13. Le système doit permettre à l'étudiant de consulter son historique, ses badges et ses performances.[scribd](#)
14. Le système doit envoyer des notifications aux utilisateurs pour les sessions, résultats ou événements importants.[scribd](#)
15. Le système doit permettre le mode Examen, le mode Challenge et le mode Entraînement.[scribd](#)

Exigences non fonctionnelles

1. Le système doit être sécurisé avec authentification, gestion des rôles et protection des données sensibles.[geeksforgeeks+1](#)
2. Le système doit être responsive et utilisable sur ordinateur, tablette et mobile.[scribd](#)
3. Le système doit offrir un temps de réponse rapide, surtout pour les quiz en direct et la correction automatique.[geeksforgeeks+1](#)
4. Le système doit être disponible de manière fiable pendant les sessions de quiz.[geeksforgeeks+1](#)
5. Le système doit garantir l'intégrité des réponses et des scores.[geeksforgeeks+1](#)
6. Le système doit être facile à utiliser pour les enseignants, les étudiants et les invités.[scribd](#)
7. Le système doit supporter un nombre important d'utilisateurs simultanés lors des quiz en direct.[scribd](#)
8. Le système doit conserver les données de sessions, réponses et résultats de manière persistante.[scribd](#)
9. Le système doit être compatible avec les navigateurs web modernes.[scribd](#)

10. Le module IA doit traiter les documents de manière fiable et permettre la validation manuelle des questions générées.[scribd](#)

Outil	Rôle dans le projet
React / Next.js	Créer l'interface web du tableau de bord, des QCM, des sessions live et des résultats.
Flutter	Créer l'application mobile Android/iOS avec une seule base de code. Flutter est adapté aux applis cross-platform et temps réel. justacademy+1
Spring Boot	Développer le backend principal, les API REST, l'authentification, la gestion des QCM, des sessions et des résultats.
WebSocket / Socket.IO / Spring WebSocket	Gérer le temps réel : questions live, réponses instantanées, classement, chronomètre. Des plateformes quiz temps réel utilisent typiquement REST + WebSocket pour cette partie. github+2
PostgreSQL	Stocker les utilisateurs, QCM, questions, réponses, sessions, participations et statistiques.
Redis	Gérer les données temporaires et rapides : rooms live, timer, leaderboard, cache.
Firebase / FCM	Envoyer les notifications push sur mobile et éventuellement les notifications in-app.
Cloud Storage	Stocker les PDF, Word, images et fichiers importés pour le module IA.
OpenAI API / LLM	Générer automatiquement des questions à partir de documents PDF/Word/PPT.
Docker	Conteneuriser le backend, la base et les services pour faciliter le déploiement.
Git + GitHub	Versioning du code, collaboration et suivi du projet.
Postman	Tester les API REST du backend.
Figma	Maquettage UI/UX avant développement.

VS Code / IntelliJ IDEA	Environnement de développement.
------------------------------------	---------------------------------

Architecture simple du projet

Tu peux organiser la réalisation comme ça :

- **Frontend web** : React / Next.js.
- **Frontend mobile** : Flutter.
- **Backend** : Spring Boot.
- **Temps réel** : WebSocket.
- **Base de données** : PostgreSQL.
- **Cache / session live** : Redis.
- **IA** : API LLM.
- **Fichiers** : stockage cloud.

Ordre de développement conseillé

1. Créer le **backend** et la base de données.
2. Faire l'**authentification** et la gestion des rôles.
3. Développer la **gestion des QCM et questions**.
4. Ajouter les **sessions live** et le **temps réel**.
5. Ajouter le **module IA** pour générer les quiz depuis les documents.
6. Développer le **frontend web**.
7. Développer l'**application mobile**.
8. Tester et déployer.

